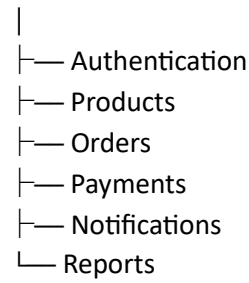
## MONOLITHIC (single repo / single application)

Single Repository



All these modules are part of one application

**PROBLEM EXAMPLE**

Suppose you want to make a small change in the Authentication module.

Even though only Auth changed:

1. The entire application must be built again.
2. The whole application must be tested.
3. The complete application must be deployed.

So, a small change affects the whole project.

**DISADVANTAGES OF MONOLITHIC APPLICAITON**

***1. Inefficient Scaling***

Suppose:

1. Authentication receives 1,000 users/day
2. Products receive 100,000 users/day

In a monolith, you cannot scale only the Product module.

Instead, you must increase the resources (CPU, RAM, servers) for the entire application, which

wastes money and resources

***2.Large Team - code conflicts***

Imagine:

1. 20 developers
2. Everyone works in the same repository.

Problems:

1. Merge conflicts
2. Developers accidentally modify each other's code
3. Longer code reviews
4. More communication overhead
5. Higher maintenance cost

As the team grows, managing one codebase becomes difficult.

***3.Bigger Project = Hard to Manage***

As features increases:

1. Auth
2. Products
3. Orders
4. Payments
5. Notifications
6. Inventory
7. Shipping
8. Reviews
9. Coupons
10. Analytics
11. Admin

The repository becomes huge.

Problems:

1. Slow builds
2. Long deployment time
3. Difficult debugging
4. Hard to find code
5. Increased maintenance effort

A very large monolithic application becomes difficult to manage.

***4. Difficult Onboarding***

When a new developer joins:

They need to understand:

1. Authentication
2. Products
3. Orders
4. Payments
5. Database
6. APIs
7. Business logic

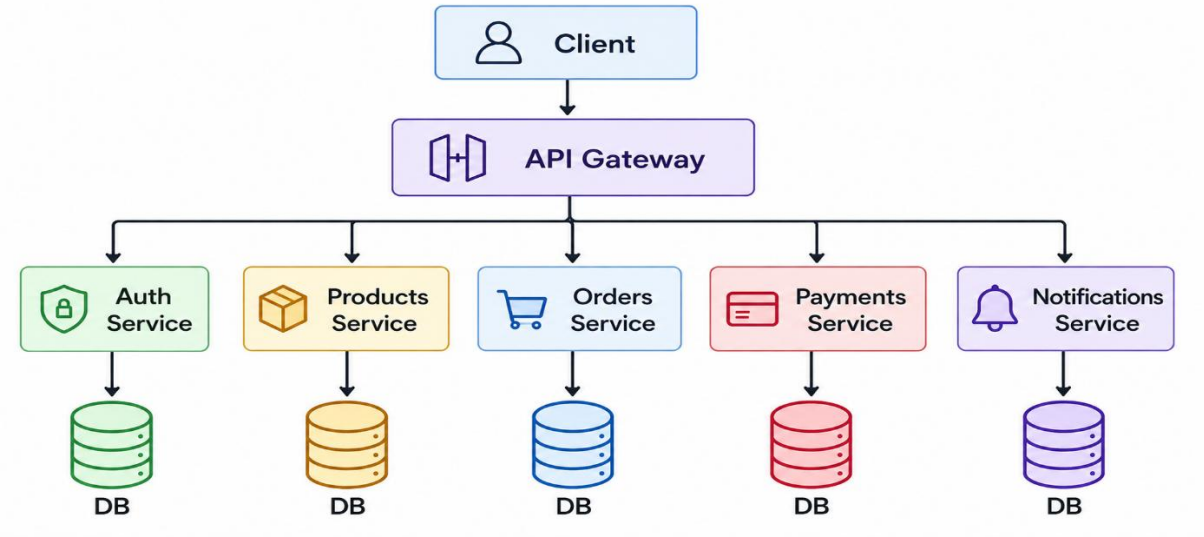
This takes a lot of time because everything is connected in one project.

Result:

1. Slower onboarding
2. Lower productivity for new team members

***5.Specific to language***

## MICROSERVICES




A microservice architecture breaks a large application into small, independent services.

Each service:

1. Has its own source code (repository)
2. Has its own database (recommended)
3. Can be developed independently
4. Can be tested independently
5. Can be deployed independently
6. Can be scaled independently

Each service performs one specific business function.

**EXAMPLE OF DEPLOYMENT**

Suppose you need to change the Authentication Service.

Only these steps are required:

1. Build Authentication Service
2. Test Authentication Service
3. Deploy Authentication Service

Other services continue running without interruption.

Result:

1. Faster deployment
2. Less risk
3. Smaller impact

***ADVANTAGES OF MICROSERVICES***

1. Independent Scaling
2. Smaller Teams
3. Easier Maintenance

4. Faster Onboarding
5. Independent Technology Stack
6. Fault Isolation

***DISADVANTAGES OF MICROSERVICES***

1. communication complexity
2. network latency
3. data management
4. Monitoring and Debugging
5. Deployment Complexity

---